A neuron from scratch

🎯 Today's goal

By the end of this hour you will:

- know what an artificial **neuron** is: inputs × **weights** + **bias**
- turn its result into a decision with an **activation function** (step and sigmoid)
- build a neuron in NumPy and make it act like the logic gates **OR** and **AND**
- let a neuron **learn** its own weights with the perceptron rule
- see why one neuron can't solve **XOR**, and why we need layers

🤔 Why this matters

Welcome to **deep learning**, the part of machine learning behind face unlock on phones, voice assistants, translation apps and chatbots. Its models are called **neural networks**: many tiny calculators called **neurons**, connected in **layers**. "Deep" just means "many layers".

The name comes from the brain. A brain cell receives signals from other cells, and if the signals together are strong enough, it "fires" and passes a signal on. An artificial neuron is a very simplified copy of that idea, made of nothing more than multiplication, addition and one small function. Over the last few days scikit-learn did the learning for you with `.fit()`. Today you'll build the smallest piece yourself and watch it learn.

🧠 Inputs, weights and a bias

Imagine a neuron that decides *"Should I go to the beach today?"*. It looks at three yes/no facts (1 = yes, 0 = no). These made-up facts are its **inputs**:

- Is it sunny? Is it warm? Is a friend coming?

Not every fact matters equally. Each input gets a **weight**: a number that says how important it is, like a volume knob for that input. A big weight means "this matters a lot"; a negative weight means "this speaks *against* going". Finally, the **bias** is a starting push that is added no matter what. A negative bias means "I'm a bit lazy: I need good reasons before I say yes".

The neuron multiplies each input by its weight, adds everything up, and adds the bias. The result is called the **weighted sum**, usually written `z`:

`z = input1 × weight1 + input2 × weight2 + input3 × weight3 + bias`

```
# Today: sunny, not warm, a friend is coming (made-up example)
inputs = [1, 0, 1]
weights = [3, 1, 2]     # sun matters most, then the friend, then warmth
bias = -2.5             # a lazy neuron: needs a good reason to say yes

z = (inputs[0] * weights[0]
     + inputs[1] * weights[1]
     + inputs[2] * weights[2]
     + bias)
print("Weighted sum z =", z)
```

```
Weighted sum z = 2.5
```

Writing out every multiplication gets long when there are 64 or 1000 inputs. NumPy (Day 9) has a shortcut: `np.dot(x, w)`, the **dot product**, multiplies the two arrays element by element and adds the results. The `@` sign does exactly the same:

```
import numpy as np

x = np.array(inputs)
w = np.array(weights)

print(x * w)                 # element by element: [3 0 2]
print(np.dot(x, w) + bias)   # multiply AND add up, then add the bias
print(x @ w + bias)          # the same thing, shorter
```

```
[3 0 2]
2.5
2.5
```

⚡ From a number to a decision: activation

`z = 2.5` is just a number. To turn it into an answer, the neuron passes it through an **activation function**. It decides how strongly the neuron "fires". The simplest one is the **step function**: if `z` is above 0, answer 1 (yes), otherwise 0 (no).

```
def step(z):
    return (z > 0) * 1     # True × 1 = 1, False × 1 = 0

print(step(2.5))           # our beach day
print(step(-0.3))
print(step(np.array([-2, -0.5, 0, 0.5, 2])))   # works on a whole array too
```

```
1
0
[0 0 0 1 1]
```

The step function is very blunt: `z = 0.01` and `z = 500` both give 1. The **sigmoid** function is gentler. It squashes any number into a value between 0 and 1, which we can read as "how sure" the neuron is. Big negative `z` → close to 0, big positive → close to 1, and `z = 0` → exactly 0.5 (can't decide). It uses `np.exp`, the "e to the power of" function from maths:

`sigmoid(z) = 1 / (1 + e^(-z))`. You don't need to remember the formula, only its S-shape.

```
def sigmoid(z):
    return 1 / (1 + np.exp(-z))

zs = np.array([-6, -2, -1, 0, 1, 2, 6])
print(zs)
print(np.round(sigmoid(zs), 3))
```

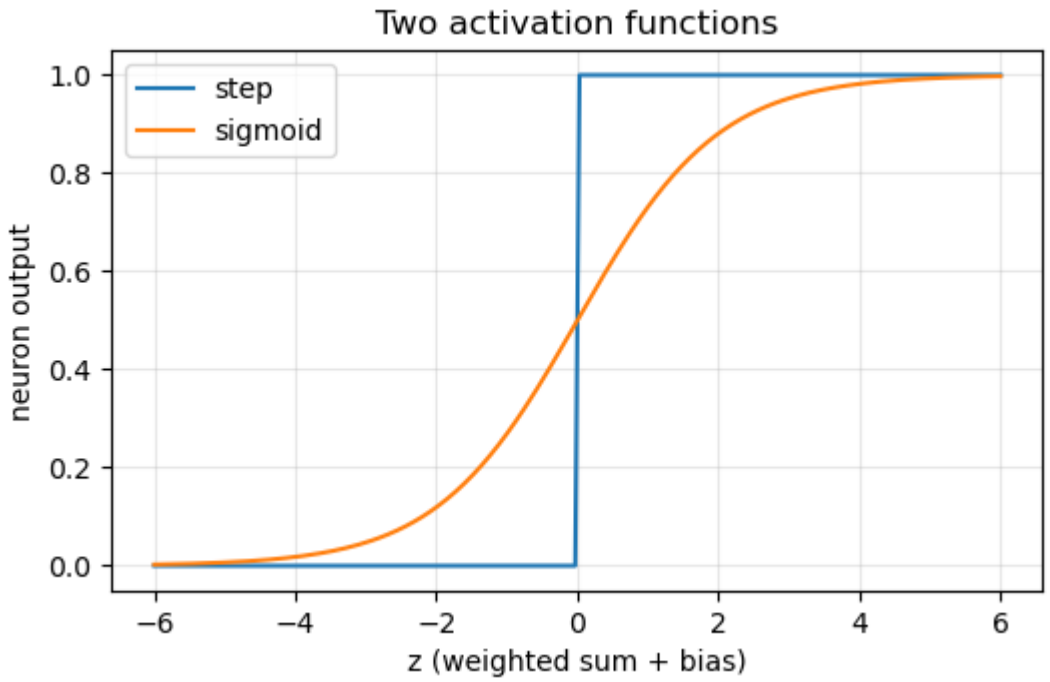
```
[-6 -2 -1  0  1  2  6]
[0.002 0.119 0.269 0.5   0.731 0.881 0.998]
```

Here are both activation functions side by side:

```
import matplotlib.pyplot as plt

z_values = np.linspace(-6, 6, 200)

plt.figure(figsize=(6, 3.5))
plt.plot(z_values, step(z_values), label="step")
plt.plot(z_values, sigmoid(z_values), label="sigmoid")
plt.xlabel("z (weighted sum + bias)")
plt.ylabel("neuron output")
plt.title("Two activation functions")
plt.legend()
plt.grid(alpha=0.3)
plt.show()
```



🧩 A whole neuron in one function

Put the pieces together: weighted sum, plus bias, through an activation. That's a neuron:

```
def neuron(x, w, b):
    z = np.dot(x, w) + b
    return sigmoid(z)

w = np.array([3, 1, 2])
b = -2.5

# Four made-up days: [sunny, warm, friend coming]
print(f"Sunny + friend:   {neuron(np.array([1, 0, 1]), w, b):.2f}")
print(f"Everything yes:   {neuron(np.array([1, 1, 1]), w, b):.2f}")
print(f"Only warm:        {neuron(np.array([0, 1, 0]), w, b):.2f}")
print(f"Nothing at all:   {neuron(np.array([0, 0, 0]), w, b):.2f}")
```

```
Sunny + friend:   0.92
Everything yes:   0.97
Only warm:        0.18
Nothing at all:   0.08
```

The neuron is 92% keen on a sunny day with a friend, but only 18% keen when it's merely warm. All of its "opinion" lives in three weights and one bias. **Learning**, in deep learning, means finding good values for those numbers.

🔌 A neuron as a logic gate

Computers are built from tiny **logic gates**. They take two yes/no inputs and give one yes/no answer:

input 1	input 2	OR (at least one)	AND (both)	XOR (exactly one)
0	0	0	0	0
0	1	1	0	1
1	0	1	0	1
1	1	1	1	0

Can one neuron with a step activation act like OR? We need weights where "at least one 1" pushes `z` above 0. Try both weights = 1 and bias = −0.5. We put all four input pairs in a 2D array `X` (one row per example) so we can check them all at once: `X @ w` does the dot product for **every row**.

```
X = np.array([[0, 0],
              [0, 1],
              [1, 0],
              [1, 1]])
y_or = np.array([0, 1, 1, 1])      # the answers we want

w_or = np.array([1, 1])
b_or = -0.5

z = X @ w_or + b_or                # 4 weighted sums at once
print("z:          ", z)
print("prediction: ", step(z))
print("wanted:     ", y_or)
```

```
z:           [-0.5  0.5  0.5  1.5]
prediction:  [0 1 1 1]
wanted:      [0 1 1 1]
```

It works! `X` has the shape `(4, 2)`: 4 rows, 2 columns. The rule for `X @ w` is: **the number of columns in X must equal the number of weights**. Give it 3 weights for 2 columns and NumPy complains:

💡 This example has a mistake on purpose. Run it and read the error.

```
w_wrong = np.array([1, 1, 1])
print(X.shape, w_wrong.shape)
X @ w_wrong
```

```
(4, 2) (3,)
```

⚠️ ValueError

The kind of value is right but the value itself isn't, like int("hello").

```
Traceback (most recent call last):
  line 3, in <module>
    X @ w_wrong
    ~~^~~~~~~~~
ValueError: matmul: Input operand 1 has a mismatch in its core dimension 0, with
gufunc signature (n?,k),(k,m?)->(n?,m?) (size 3 is different from 2)
```

Read the last line of the error: *"size 3 is different from 2"*. Shape mismatches are the most common error in deep learning. When you see one, print the `.shape` of everything involved.

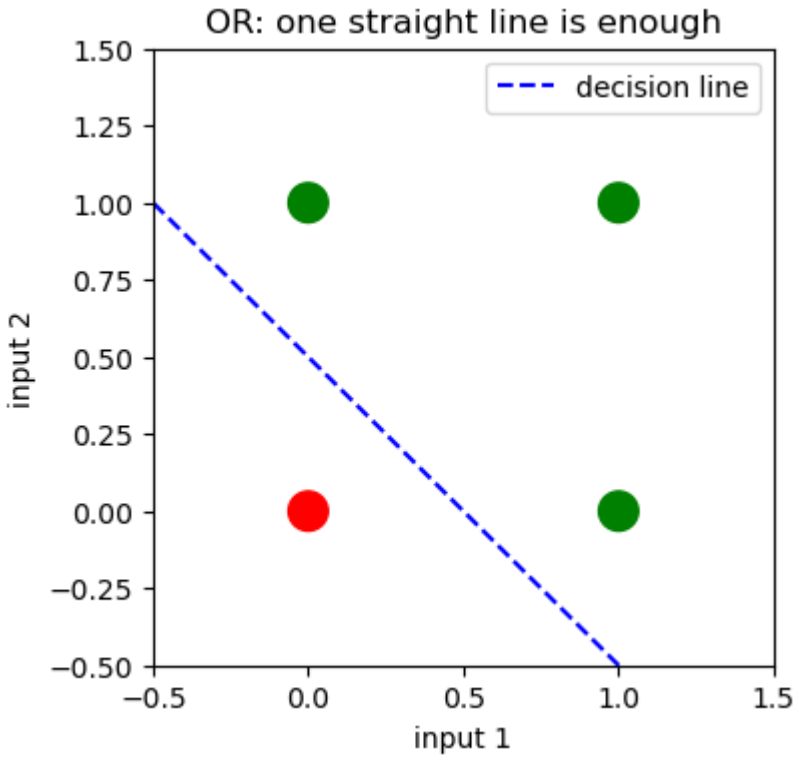
Seeing the decision line

A neuron with two inputs splits the plane with a **straight line**: the points where `z` is exactly 0. On one side it says 1, on the other side 0. Let's draw the OR neuron's line (red = 0, green = 1):

```
colours = ["red" if label == 0 else "green" for label in y_or]

# The line where w1·x1 + w2·x2 + b = 0, rearranged to find x2
x1_line = np.linspace(-0.5, 1.5, 10)
x2_line = -(w_or[0] * x1_line + b_or) / w_or[1]

plt.figure(figsize=(4, 4))
plt.scatter(X[:, 0], X[:, 1], c=colours, s=200)
plt.plot(x1_line, x2_line, "b--", label="decision line")
plt.xlim(-0.5, 1.5)
plt.ylim(-0.5, 1.5)
plt.xlabel("input 1")
plt.ylabel("input 2")
plt.title("OR: one straight line is enough")
plt.legend()
plt.show()
```



🎓 Letting the neuron learn: the perceptron rule

Picking weights by hand is fine for 2 inputs, impossible for 1000. So let the neuron find them itself. A neuron with a step activation is called a **perceptron** (invented in 1958), and it learns like someone adjusting a shower: too cold → turn a little warmer, too hot → a little colder, just right → leave it.

1. Start with all weights and the bias at 0 (it knows nothing).
2. For each example: make a prediction and compute the **error** = wanted − predicted. It is `+1` ("you should have said yes"), `−1` ("you should have said no") or `0` (correct).
3. Nudge each weight: `w += learning_rate × error × input`, and the bias: `b += learning_rate × error`. Inputs that were 0 didn't cause the mistake, so their weights don't change.
4. Repeat for all examples, many times.

Two new words:

- **Learning rate** (`lr`): how big each nudge is. Small = careful, slow steps.
- **Epoch**: one full pass through all the training examples. Training usually takes many epochs.

Let's teach a perceptron **AND**. `zip(X, y_and)` walks through the rows of `X` and the answers in `y_and` side by side, one pair at a time:

```
y_and = np.array([0, 0, 0, 1])

w = np.zeros(2)      # start knowing nothing
b = 0.0
lr = 0.1             # learning rate

for epoch in range(1, 9):
    mistakes = 0
    for x, target in zip(X, y_and):
        prediction = step(np.dot(x, w) + b)
        error = target - prediction       # +1, 0 or -1
        w += lr * error * x
        b += lr * error
        if error != 0:
            mistakes += 1
    print(f"epoch {epoch}: mistakes = {mistakes}  w = {np.round(w, 2)}  b = {b:.2f}")

print("Predictions:", step(X @ w + b), " wanted:", y_and)
```

```
epoch 1: mistakes = 1  w = [0.1 0.1]  b = 0.10
epoch 2: mistakes = 3  w = [0.2 0.1]  b = 0.00
epoch 3: mistakes = 3  w = [0.2 0.1]  b = -0.10
epoch 4: mistakes = 2  w = [0.2 0.2]  b = -0.10
epoch 5: mistakes = 1  w = [0.2 0.1]  b = -0.20
epoch 6: mistakes = 0  w = [0.2 0.1]  b = -0.20
epoch 7: mistakes = 0  w = [0.2 0.1]  b = -0.20
epoch 8: mistakes = 0  w = [0.2 0.1]  b = -0.20
Predictions: [0 0 0 1]  wanted: [0 0 0 1]
```

Watch the mistakes fall to 0 by epoch 6. From then on every error is 0, so nothing changes: the neuron has learned AND by itself. Let's wrap the training loop in a reusable function and try it on both gates:

```
def train_perceptron(X, y, epochs=10, lr=0.1):
    w = np.zeros(X.shape[1])      # one weight per column of X
    b = 0.0
    for epoch in range(epochs):
        for x, target in zip(X, y):
            error = target - step(np.dot(x, w) + b)
            w += lr * error * x
            b += lr * error
    return w, b

for name, y in [("AND", y_and), ("OR", y_or)]:
    w, b = train_perceptron(X, y)
    print(f"{name}: wanted {y}, learned {step(X @ w + b)}")
    print(f"     weights {np.round(w, 2)}, bias {b:.2f}")
```

```
AND: wanted [0 0 0 1], learned [0 0 0 1]
     weights [0.2 0.1], bias -0.20
OR: wanted [0 1 1 1], learned [0 1 1 1]
     weights [0.1 0.1], bias 0.00
```

× The problem one neuron can't solve: XOR

XOR means "exactly one of the two". It sounds just as easy as OR. Let's give the perceptron 1000 epochs:

```
y_xor = np.array([0, 1, 1, 0])

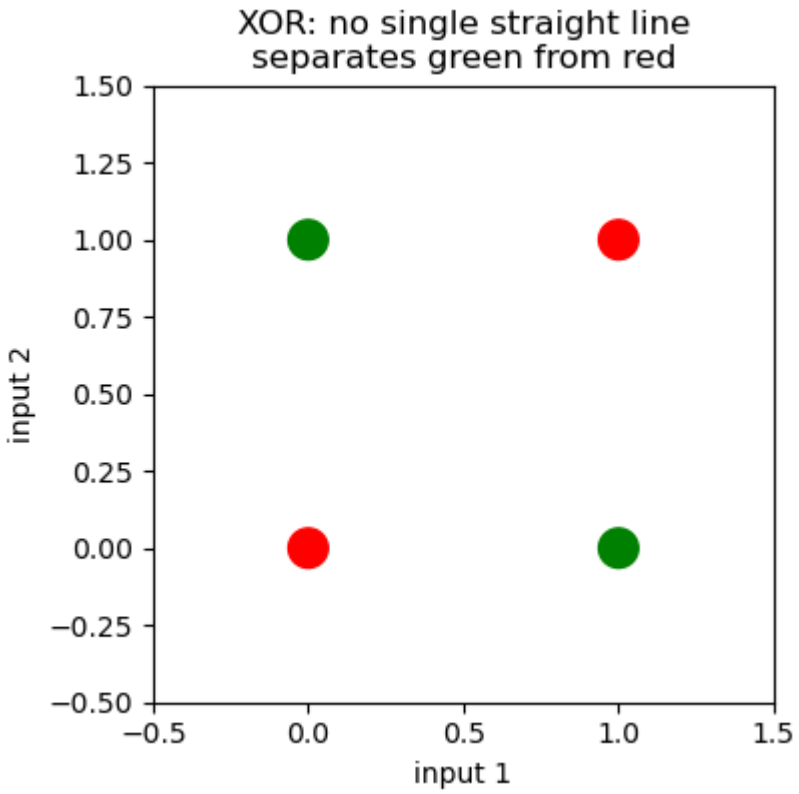
w, b = train_perceptron(X, y_xor, epochs=1000)
print("wanted: ", y_xor)
print("learned:", step(X @ w + b))
print("correct:", (step(X @ w + b) == y_xor).sum(), "out of 4")
```

```
wanted:  [0 1 1 0]
learned: [1 1 0 0]
correct: 2 out of 4
```

It never gets all four right. The picture shows why:

```
colours = ["red" if label == 0 else "green" for label in y_xor]

plt.figure(figsize=(4, 4))
plt.scatter(X[:, 0], X[:, 1], c=colours, s=200)
plt.xlim(-0.5, 1.5)
plt.ylim(-0.5, 1.5)
plt.xlabel("input 1")
plt.ylabel("input 2")
plt.title("XOR: no single straight line\nseparates green from red")
plt.show()
```



The green points sit on one diagonal and the red points on the other. Try to imagine one straight line with all greens on one side and all reds on the other: it can't be done. And one neuron can only draw *one straight line*.

The fix: use several neurons in a row. A first **layer** of neurons draws a few lines, and a second layer combines their answers. That's a neural network, and you'll build one that solves XOR tomorrow. (In 1969 a famous book pointed out this XOR limit, and interest in neural networks dropped for years until layered networks became trainable.)

⚠️ Common mistakes

- Forgetting the bias. Without it, the neuron's line must pass through (0, 0), and a gate like AND becomes impossible.

- Shape mismatch in `X @ w`: the number of columns of `X` must equal the number of weights. Print the `.shape`s.
- Writing `np.exp(z)` instead of `np.exp(-z)` in the sigmoid: big `z` then gives values near 0 instead of 1.
- Using `=` instead of `+=` in the learning rule: the weights forget everything they learned before.
- Expecting one neuron to learn anything. It can only separate data with one straight line.

✅ Recap

- A neuron computes `z = inputs · weights + bias`, then an activation: `output = activation(z)`.
- **Weights** say how much each input matters; the **bias** is a starting push.
- `np.dot(x, w)` or `x @ w` is the weighted sum; `X @ w` does it for every row of `X`.
- **Step** gives 0 or 1; **sigmoid** `1 / (1 + np.exp(-z))` gives a smooth value between 0 and 1.
- The **perceptron rule**: `w += lr × error × x`, `b += lr × error`, repeated for many **epochs**.
- One neuron draws one straight line, so it can learn AND and OR but not XOR. That's why networks have layers.

🏋️ Exercises

Write your answer in the space under each exercise. Try it on the computer too, by opening this day's `index.html`. Solutions are in the answer key at the end.

1. Weighted sum

A made-up neuron has three inputs `x`, three weights `w` and a bias `b`. Calculate its weighted sum plus bias with `np.dot` (or `@`) and store it in a variable called `z`.

```
import numpy as np

x = np.array([3.0, 7.0, 1.0])
w = np.array([0.5, 0.2, -1.0])
b = 0.4

# Calculate z
```

Hints:

1. `np.dot(x, w)` multiplies the pairs and adds them up.
2. Don't forget to add the bias at the end.
3. `z = np.dot(x, w) + b`

2. Write the sigmoid

Finish the function `sigmoid(z)` so it returns `1 / (1 + e^-z)`, using `np.exp`. It must work for a single number and for a whole array.

```
import numpy as np

def sigmoid(z):
    # Replace the line below
    return z

print(sigmoid(0))     # should be 0.5
```

Hints:

1. `np.exp(-z)` is e to the power of minus z.
2. Mind the brackets: the 1 + … goes under the line, so it needs its own brackets.
3. `return 1 / (1 + np.exp(-z))`

3. Build an AND gate by hand

In the lesson, weights `[1, 1]` and bias `−0.5` made an OR gate. Now choose your own weights `w` and bias `b` so that the neuron with a step activation acts like **AND**: the answer is 1 only when *both* inputs are 1.

```
import numpy as np

def step(z):
    return (z > 0) * 1

X = np.array([[0, 0], [0, 1], [1, 0], [1, 1]])
y_and = np.array([0, 0, 0, 1])

# Change these numbers
w = np.array([1, 1])
b = 0

print("prediction:", step(X @ w + b))
print("wanted:    ", y_and)
```

Hints:

1. With weights `[1, 1]`, `X @ w` is 0, 1, 1 and 2. Only the last one should end up above 0.
2. So the bias must pull 1 below zero but leave 2 above zero.
3. Keep `w = np.array([1, 1])` and use `b = -1.5`.

4. Teach a perceptron OR

The training loop below is missing its two most important lines: the updates. Add the perceptron rule so that, after 10 epochs, the neuron has learned **OR** by itself. Don't change the starting weights.

```
import numpy as np

def step(z):
    return (z > 0) * 1

X = np.array([[0, 0], [0, 1], [1, 0], [1, 1]])
y_or = np.array([0, 1, 1, 1])

w = np.zeros(2)
b = 0.0
lr = 0.1

for epoch in range(10):
    for x, target in zip(X, y_or):
        prediction = step(np.dot(x, w) + b)
        error = target - prediction
        # Update w and b here (two lines)

print("learned:", step(X @ w + b))
```

Hints:

1. Both new lines go inside the inner loop, indented like `error = …`.
2. The weights change by learning rate × error × input. The bias changes by learning rate × error.
3. `w += lr * error * x` and `b += lr * error`

5. Challenge: a neuron for many rows at once

A made-up "does this plant need water?" neuron looks at three numbers per plant: hours of sun, days since the last watering, and soil dryness (0 to 1). `X` holds 5 plants, one per row.

- Store the sigmoid output for **all 5 plants at once** in `probs` (no loop: use `X @ w`).
- Store the decisions in `decisions`: 1 where `probs` is above 0.5, otherwise 0.

```
import numpy as np

def sigmoid(z):
    return 1 / (1 + np.exp(-z))

X = np.array([[8, 3, 0.9],
              [2, 1, 0.2],
              [6, 4, 0.7],
              [1, 0, 0.1],
              [5, 2, 0.5]])
w = np.array([0.3, 0.8, 2.0])
b = -4.0

# Calculate probs and decisions
```

Hints:

1. `X @ w + b` gives 5 weighted sums, one per plant.
2. `probs = sigmoid(X @ w + b)`. Then `probs > 0.5` gives True/False for each plant.
3. Turn True/False into 1/0 by multiplying by 1: `decisions = (probs > 0.5) * 1`

➡️ Tomorrow

Gradient descent and a 2-layer network: the perceptron rule only works for one neuron with a step. Tomorrow you'll learn how real networks learn: measure how wrong they are with a *loss*, then walk "downhill" step by step with *gradient descent*. Then you'll stack neurons into two layers and finally solve XOR.

🔑 Answer key

Try the exercises yourself first! There is often more than one correct answer.

1. Weighted sum

```
import numpy as np

x = np.array([3.0, 7.0, 1.0])
w = np.array([0.5, 0.2, -1.0])
b = 0.4

z = np.dot(x, w) + b
print(z)
```

2. Write the sigmoid

```
import numpy as np

def sigmoid(z):
    return 1 / (1 + np.exp(-z))

print(sigmoid(0))     # should be 0.5
```

3. Build an AND gate by hand

```
import numpy as np

def step(z):
    return (z > 0) * 1

X = np.array([[0, 0], [0, 1], [1, 0], [1, 1]])
y_and = np.array([0, 0, 0, 1])

w = np.array([1, 1])
b = -1.5

print("prediction:", step(X @ w + b))
print("wanted:    ", y_and)
```

4. Teach a perceptron OR

```
import numpy as np

def step(z):
    return (z > 0) * 1

X = np.array([[0, 0], [0, 1], [1, 0], [1, 1]])
y_or = np.array([0, 1, 1, 1])

w = np.zeros(2)
b = 0.0
lr = 0.1

for epoch in range(10):
    for x, target in zip(X, y_or):
        prediction = step(np.dot(x, w) + b)
        error = target - prediction
        w += lr * error * x
        b += lr * error

print("learned:", step(X @ w + b))
```

5. Challenge: a neuron for many rows at once

```
import numpy as np

def sigmoid(z):
    return 1 / (1 + np.exp(-z))

X = np.array([[8, 3, 0.9],
              [2, 1, 0.2],
              [6, 4, 0.7],
              [1, 0, 0.1],
              [5, 2, 0.5]])
w = np.array([0.3, 0.8, 2.0])
b = -4.0

probs = sigmoid(X @ w + b)
decisions = (probs > 0.5) * 1
print(np.round(probs, 2))
print(decisions)
```